

DEMANDIQ

AI-Enhanced Supply Chain Forecasting and Inventory Management System

Project Documentation Report

Submitted by: Keerthi Sayani

Roll Number: _____

Supervisor/Guide Name: _____

Designation: _____

University Name: _____

Department Name: _____

Academic Year: 2025-2026

Declaration by the Student

I hereby declare that the project work entitled "DemandIQ: AI-Enhanced Supply Chain Forecasting and Inventory Management System" submitted to _____ University, in partial fulfillment of the requirements for the degree program, is a record of original work carried out by me under the guidance of _____ and has not been submitted elsewhere for any other degree or diploma.

Student Name: Keerthi Sayani

Signature: _____

Date: 25 March 2026

Certificate

This is to certify that the project titled "DemandIQ: AI-Enhanced Supply Chain Forecasting and Inventory Management System" is a bonafide work carried out by Keerthi Sayani under the guidance and supervision of the undersigned. The project has been completed successfully in partial fulfillment of the academic requirements of the degree program.

Student Name: Keerthi Sayani

Roll Number: _____

Guide Name: _____

Guide Signature: _____

Head of Department: _____

HOD Signature: _____

Date: 25 March 2026

Acknowledgment

I express my sincere gratitude to my project guide for continuous encouragement, valuable suggestions, and technical guidance throughout the development of this project. I also thank the Head of the Department and faculty members for their support and motivation during the course of this work.

I am thankful to the lab staff and my friends for their assistance, ideas, and cooperation. I also extend my gratitude to my family for their constant support and encouragement, which helped me complete this project successfully.

Abstract

DemandIQ is an AI-enhanced supply chain forecasting and inventory management system developed to improve demand planning, inventory analysis, and business decision support in a retail environment. The project addresses the challenge of predicting future product demand accurately while also providing role-based access, dataset upload support, forecast history tracking, and natural-language interaction through an AI assistant.

The system is implemented using Python, FastAPI, HTML, CSS, JavaScript, SQLite, pandas, NumPy, and scikit-learn. The forecasting engine uses a Linear Regression model with lag features, rolling statistics, seasonal calendar features, and holiday-aware adjustments to generate short-term demand forecasts. The application also integrates OpenAI using the 'gpt-4o-mini' model to support product and supply-chain related chat interactions. If the external AI service is unavailable, the project falls back to a dataset-based local response mechanism.

The implementation includes user login, session-based authentication, temporary tokens for email verification and password reset, administrative upload workflows, inventory stock updates, and interactive dashboards. The project demonstrates how machine learning, data processing, and web application development can be integrated into a practical supply chain solution. Future scope includes model comparison, stronger deployment security, advanced analytics, and production-scale database support.

Table of Contents

1. Cover Page
2. Declaration by the Student
3. Certificate
4. Acknowledgment
5. Abstract
6. Table of Contents
7. List of Figures
8. List of Tables
9. List of Abbreviations
10. Chapter 1: Introduction
11. Chapter 2: Literature Review / Existing System
12. Chapter 3: System Analysis

13. Chapter 4: System Design
14. Chapter 5: Implementation
15. Chapter 6: Testing
16. Chapter 7: Results and Discussion
17. Chapter 8: Conclusion and Future Work
18. References / Bibliography
19. Appendices

List of Figures

1. System Architecture of DemandIQ
2. Authentication and Session Workflow
3. Forecasting Pipeline Workflow
4. AI Chat Request Workflow
5. Admin Upload and Approval Workflow

List of Tables

1. Technology Stack
2. Functional Requirements
3. Non-Functional Requirements
4. Dataset Fields
5. Core Database Tables
6. Major API Endpoints
7. Sample Test Cases

List of Abbreviations

- AI: Artificial Intelligence
- API: Application Programming Interface
- CSV: Comma-Separated Values
- DB: Database
- DFD: Data Flow Diagram
- ER: Entity Relationship
- HTML: HyperText Markup Language
- HTTP: HyperText Transfer Protocol
- ML: Machine Learning
- RMSE: Root Mean Square Error
- UAT: User Acceptance Testing
- UI: User Interface

Chapter 1: Introduction

Overview of the Topic

Demand forecasting and inventory management are critical activities in supply chain operations. Businesses require accurate estimates of future demand in order to maintain stock availability, reduce waste, avoid overstocking, and improve procurement planning. With the availability of historical retail data and machine learning methods, forecasting systems can now be integrated into web applications for practical decision support.

DemandIQ is a web-based forecasting and inventory analysis platform that combines machine learning, secure user access, administrative controls, and AI-assisted interaction. It is designed to help users monitor retail demand, generate forecasts, manage uploaded data, and ask natural-language questions related to products and supply-chain performance.

Problem Statement

Many organizations still depend on fragmented tools such as spreadsheets, manual reports, or isolated scripts for demand planning. These methods are inefficient, error-prone, and difficult to scale. They often do not provide secure access control, historical traceability, or interactive analytics support.

The problem addressed by this project is the lack of a unified platform that can perform forecasting, maintain inventory-related insights, support role-based access, and deliver user-friendly analytical interaction.

Purpose and Objectives of the Project

The project was developed with the following objectives:

- to build a web-based supply chain forecasting system,
- to forecast demand using historical retail data,
- to support inventory and dataset management,
- to provide role-based authentication and session handling,
- to integrate AI chat assistance for product and forecast queries,
- to maintain forecast history for reporting and review.

Scope of the Project

The system is intended for retail demand forecasting and inventory analysis use cases. It supports short-term forecasting, user login, administrative uploads, and dashboard-based visualization. It can be used in academic demonstrations, prototype business analysis, and small-scale forecasting workflows.

Limitations and Assumptions

- The current project uses SQLite, which is suitable for small to medium prototype workloads.
- The forecasting model is based primarily on Linear Regression and does not yet compare multiple algorithms automatically.
- Secure production deployment settings such as HTTPS-only cookie handling can be strengthened further.
- The AI chat component depends on API availability for live OpenAI responses.

Chapter 2: Literature Review / Existing System

Review of Similar Applications, Tools, or Research

Retail forecasting systems typically combine historical sales trends, seasonality, promotions, and pricing information to estimate future demand. Existing approaches range from spreadsheet-based planning tools to dedicated analytics platforms and machine learning pipelines. Many modern systems also include dashboard reporting and role-based

access.

Research and industrial tools often focus separately on either forecasting models or inventory visibility. In many student and prototype systems, forecasting remains notebook-based and disconnected from user authentication, workflow control, and live business interaction.

Analysis of Existing Systems and Their Limitations

Existing systems commonly face the following limitations:

- lack of integration between forecasting and user-facing applications,
- poor support for role-based workflows,
- limited traceability for uploads and forecasting history,
- weak support for secure password reset and verification workflows,
- minimal natural-language support for interacting with business data.

Gap Analysis

DemandIQ addresses these gaps by combining:

- forecasting,
- inventory management,
- authentication,
- administrative upload control,
- AI chat interaction,
- and historical storage

within one unified application.

Chapter 3: System Analysis

Requirements Specification

Functional Requirements

The system shall:

- allow users to log in and log out securely,
- generate category-based demand forecasts,
- store forecast history,
- provide AI chat support for dataset-aware questions,
- support admin upload of datasets and inventory files,
- manage users and roles,
- provide password reset and email verification token flows,
- display dashboards and reporting pages.

Non-Functional Requirements

The system should provide:

- usability through simple web interfaces,
- security through session-based authentication and controlled tokens,
- performance suitable for interactive forecasting and dashboard usage,
- maintainability through modular forecasting and utility functions,
- reliability through fallback chat behavior when external AI is unavailable.

Feasibility Study

Technical Feasibility

The project is technically feasible because it uses widely available technologies such as Python, FastAPI, SQLite, pandas, and scikit-learn. The required forecasting and authentication workflows can be implemented with standard libraries and web development practices.

Economic Feasibility

The system is economically feasible for academic and prototype use because it depends mainly on open-source technologies. Infrastructure cost is low, apart from optional OpenAI API usage for live chat responses.

Operational Feasibility

The system is operationally feasible because it supports direct user interaction through a browser-based interface and includes separate roles for admin, inventory manager, and viewer.

System Environment

Hardware Requirements

- Computer or laptop with standard processor
- Minimum 4 GB RAM recommended
- Internet access for OpenAI-enabled chat

Software Requirements

- Operating System: Windows or equivalent
- Python 3.x
- FastAPI environment
- Browser for frontend access

- SQLite database support

Use Case and Flow Summary

Main use cases include:

- user login and logout,
- forecast generation,
- forecast history review,
- AI chat requests,
- admin dataset upload,
- inventory upload,
- user and role management.

Chapter 4: System Design

Architectural Design

The project follows a layered architecture with frontend, backend, and data-storage components.

- Frontend pages provide the visual interface for login, dashboards, reports, admin controls, and AI chat.
- The FastAPI backend handles routing, authentication, forecast generation, uploads, and AI integration.
- SQLite databases store authentication data and forecast history.
- CSV and uploaded files provide source data for forecasting and inventory workflows.

Data Flow Overview

The major data flows are:

1. User credentials flow from login form to backend verification and session creation.
2. Forecast request flow from dashboard to forecasting engine and back to results page.
3. AI chat flow from user message to context construction, OpenAI request, and response delivery.
4. Admin upload flow from selected file to validation, storage, retraining, and status summary.

Database Design

The project uses the following important database entities:

- 'users'
- 'auth_sessions'
- 'auth_tokens'
- 'auth_audit_logs'
- 'upload_metadata'
- 'inventory_stock'
- 'forecast_history'

Key Table Purposes

- 'users' stores account and role information.
- 'auth_sessions' stores active session records associated with 'session_token'.
- 'auth_tokens' stores one-time temporary tokens for email verification and password reset.
- 'forecast_history' stores generated forecast metadata and results.
- 'inventory_stock' stores uploaded stock information.

UI Design

The application contains separate interfaces for:

- login page,
- dashboard page,
- results dashboard,
- admin dashboard,
- AI chat page.

Chapter 5: Implementation

Tools and Technologies Used

Front-end

- HTML
- CSS
- JavaScript

Back-end

- Python
- FastAPI

Database

- SQLite

Platform

- Web application

Modules and Components Description

Backend API Module

The core backend is implemented in 'forecast_api.py'. It manages routing, authentication, sessions, tokens, role checks, forecasting endpoints, upload processing, and AI chat requests.

Forecasting Module

The forecasting logic is implemented in 'forecasting_core.py'. It uses Linear Regression with lag features, rolling statistics, calendar seasonality, and holiday-aware adjustments.

Preprocessing Module

The preprocessing logic supports data cleaning and preparation for consistent model inputs.

Model Training Module

The model training and evaluation logic calculates metrics such as MAE, MSE, RMSE, and R2 score to assess predictive quality.

Frontend Module

The frontend pages allow users to access system functionality visually and

interactively.

Dataset and Feature Engineering

The dataset contains 164,400 records with fields such as:

- date,
- store id,
- product id,
- category,
- inventory level,
- units sold,
- units ordered,
- demand forecast,
- price,
- discount,
- weather condition,
- holiday or promotion,
- competitor pricing,
- seasonality.

The forecasting engine constructs features such as:

- lag_1,
- lag_7,
- lag_14,
- lag_28,
- rolling_mean_7,
- rolling_mean_14,
- rolling_std_7,
- day-of-week and month features,
- cyclic seasonal encodings.

Implementation Workflow

1. Dataset is loaded and cleaned.
2. Historical daily series are built for categories or products.
3. Features are engineered from lag and seasonal patterns.
4. Linear Regression is trained on prepared data.
5. Forecasts are generated for the required horizon.
6. Results are stored in forecast history.
7. Dashboards and AI chat consume the processed data.

Screens and Interfaces

The implementation includes:

- login screen,
- dashboard screen,
- results screen,
- admin dashboard,
- AI chat screen.

Chapter 6: Testing

Testing Methods

Unit Testing

Individual utility functions, validation logic, and forecasting helpers can be verified

independently.

Integration Testing

The interaction between login, session handling, forecasting endpoints, upload workflows, and AI chat can be validated through integrated requests.

System Testing

The complete application can be tested by verifying user login, forecast generation, dashboard access, uploads, and AI responses in sequence.

User Acceptance Testing

Users can validate whether the dashboard, forecast pages, and chat responses meet expected usability and functional goals.

Sample Test Cases

1. Valid login should create a session and redirect the user to the dashboard.
2. Invalid login should return an authentication error.
3. Forecast request for a category should return history and forecast values.
4. Logout should clear the active 'session_token'.
5. Password reset token should expire if not used within its validity period.
6. Inventory upload should update stock records and return alert summaries.

7. AI chat should answer using OpenAI or fallback local dataset logic.

Tools Used for Testing

- Browser-based manual testing
- API request testing through backend endpoints
- Built-in model evaluation metrics in Python

Chapter 7: Results and Discussion

Output Summary

The system successfully combines forecasting, user authentication, upload workflows, and AI-assisted interaction in a single web application. Forecast output is available by category, and historical demand context is shown alongside predicted values.

Project Demonstration Summary

The project demonstrates the following:

- a working login and role-based system,
- dashboard-based forecasting,
- forecast history persistence,
- administrative control over uploads and users,
- inventory stock tracking,
- AI chat integration using 'gpt-4o-mini'.

Performance Metrics

The project includes evaluation support through:

- MAE,
- MSE,
- RMSE,
- R2 score.

These metrics help assess the quality of the forecasting model.

Comparative Discussion

Compared with simple manual systems or disconnected notebook-based implementations, DemandIQ provides better integration, traceability, user control, and accessibility. It improves the practical usability of forecasting for academic and business demonstration purposes.

Chapter 8: Conclusion and Future Work

Summary of Achievements

DemandIQ successfully implements a practical supply chain forecasting system with:

- machine learning based demand prediction,
- secure session management,
- temporary token-based account workflows,
- historical forecast storage,
- inventory upload handling,
- AI-assisted analysis.

Limitations of the Current Work

- backend responsibilities are concentrated in a large core file,
- database scalability is limited by SQLite,
- deployment security can be strengthened further,
- the forecasting approach can be expanded with more models and comparison strategies.

Suggestions for Future Enhancement

- migrate to a production-grade database such as PostgreSQL,
- modularize backend services,
- add HTTPS-ready secure cookie handling,
- support multiple forecasting models,
- add email delivery for verification and reset flows,
- improve dashboards with more advanced KPIs and reports,
- support cloud deployment and monitoring.

References / Bibliography

1. FastAPI Documentation
2. scikit-learn Documentation
3. pandas Documentation
4. NumPy Documentation
5. SQLite Documentation
6. OpenAI API Documentation
7. General reference materials on demand forecasting and inventory management

Appendices

Appendix A: Installation Guide

The project requires Python, backend dependencies, the dataset, and a browser to run the frontend interface. Environment configuration includes an OpenAI API key for live AI chat support.

Appendix B: User Manual

Basic usage flow:

1. Start the backend server.
2. Open the login page.
3. Sign in using a valid role account.
4. Generate forecasts or open reports.
5. Use AI chat for product and demand-related questions.
6. For admin users, upload datasets or inventory files and manage users.

Appendix C: Source Code Note

The project source code is maintained separately in the implementation repository and includes backend modules, frontend pages, dataset files, and upload directories.

Appendix D: Optional Presentation and Demo

Project presentation slides, screen captures, and demo walkthrough material may be attached separately if required by the institution.